

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI Casablanca

Filière : Ingénierie Informatique et Réseaux – 3IIR
Année universitaire : 2025/2026

RAPPORT DE PROJET DE FIN D'ANNÉE

FakeFinder

Système Intelligent de Détection de Phishing
par Apprentissage Automatique

Réalisé par :

Yahya KHALIFI
Elias CHAKROUN

Tuteur académique :

Mme Soukayna MAYARA

Date de soutenance :

Juin 2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos remerciements s'adressent en premier lieu à notre encadrante, **Madame Soukayna MAYARA**, pour ses conseils avisés, ses orientations pertinentes, sa disponibilité et son suivi rigoureux tout au long de ce projet de fin d'année.

Nous remercions également le corps professoral et l'administration de **l'École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la qualité de l'enseignement dispensé et les ressources mises à notre disposition pour mener à bien nos études.

Enfin, nous adressons nos remerciements les plus chaleureux à nos familles et à nos amis pour leur soutien moral indéfectible et leurs encouragements durant tout notre cursus universitaire.

Résumé

Le phishing est l'une des cybermenaces les plus répandues, causant chaque année des pertes financières considérables. Ce projet, baptisé **FakeFinder**, propose une solution web intelligente de détection automatique d'emails frauduleux, combinant l'apprentissage automatique et une interface utilisateur moderne. Le système repose sur un modèle de classification entraîné sur un corpus de 18 650 emails labélisés. Le pipeline d'analyse combine des caractéristiques textuelles (TF-IDF) avec des caractéristiques structurelles (URLs, mots-clés). Le modèle Random Forest atteint une précision de **97%** et un ROC-AUC de **0,9928**. L'interface, développée avec le framework Django, permet à tout utilisateur de soumettre un fichier .eml et d'obtenir un verdict instantané avec un score de risque (SAFE, SUSPECT, DANGER) sans conservation des données sur le serveur.

Mots-clés : phishing, détection, apprentissage automatique, Random Forest, TF-IDF, Django, cybersécurité.

Abstract

Phishing is one of the most widespread cyberthreats, causing significant financial losses every year. This project, named **FakeFinder**, proposes an intelligent web-based solution for automatic detection of fraudulent emails, combining machine learning and a modern user interface. The system relies on a classification model trained on a corpus of 18,650 labeled emails. The analysis pipeline combines text features (TF-IDF) with structural features (URLs, keywords). The Random Forest model achieves an accuracy of **97%** and a ROC-AUC of **0.9928**. The interface, built with the Django framework, allows any user to submit an .eml file and instantly receive a risk verdict (SAFE, SUSPECT, DANGER) without retaining any user data on the server.

Keywords : phishing, detection, machine learning, Random Forest, TF-IDF, Django, cybersecurity.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des figures	vi
Liste des tableaux	vii
Introduction générale	1
1 Présentation de l'école et du contexte	2
1.1 Présentation de l'organisme d'accueil : l'EMSI	2
1.2 Le phishing : définition et enjeux	3
1.3 Technologies utilisées	4
2 Présentation du projet	5
2.1 Problématique	5
2.2 Objectifs du projet	5
2.3 Cahier des charges et fonctionnalités	5
2.4 Planning et gestion de projet (Méthode Agile)	6
3 Analyse et conception	7
3.1 Analyse des besoins	7
3.2 Conception UML	7
3.2.1 Diagramme de cas d'utilisation	7
3.2.2 Modèle de données et Diagramme de classes	7
4 Réalisation et développement	9
4.1 Environnement de développement	9
4.2 Développement du moteur de Parsing (Backend)	9
4.3 Développement du modèle de Machine Learning	9
4.4 Interface Utilisateur (Frontend)	11

5 Résultats, tests et validation	12
5.1 Évaluation du modèle de Machine Learning	12
5.2 Tests fonctionnels et de robustesse de l'application	12
5.3 Analyse critique	13
Conclusion et perspectives	14
Bibliographie	15
A Extraits de code source	16

Table des figures

1.1	Infrastructures de l'EMSI	3
3.1	Diagramme de cas d'utilisation du système	7
3.2	Diagramme de classes du système Django	8
4.1	Extrait de l'environnement de développement	10
4.2	Extrait de l'environnement de développement	10
4.3	Interface web : Zone d'upload sécurisée	11
4.4	Interface web : Zone de résultat du rapport	11

Liste des tableaux

2.1	Planning de réalisation du PFA (Avril - Mai 2026)	6
3.1	Structure de l'entité ScanReport	8
5.1	Rapport de classification du modèle Random Forest	12

Introduction générale

Dans un monde de plus en plus connecté, la messagerie électronique reste l'un des vecteurs d'attaque privilégiés des cybercriminels. Le **phishing** — technique consistant à usurper l'identité d'une organisation légitime pour soutirer des informations sensibles à ses victimes — représentait récemment plus de 36% des violations de données recensées mondialement.

Les solutions antivirales traditionnelles, basées sur des listes noires et des signatures statiques, peinent à détecter les campagnes de phishing modernes qui s'adaptent rapidement pour contourner les filtres. L'apprentissage automatique (Machine Learning), en revanche, permet d'extraire des modèles complexes à partir de données réelles et de généraliser la détection à des menaces inédites.

Ce projet a pour objectifs principaux de :

- Développer un modèle d'intelligence artificielle capable de classer les emails en phishing ou légitimes avec une haute précision.
- Intégrer ce modèle dans une application web accessible, performante et ergonomique.
- Fournir aux utilisateurs un rapport détaillé avec un score de risque et les indices détectés, tout en garantissant la confidentialité des données traitées.

Ce rapport s'articule autour de cinq chapitres. Le premier chapitre présente l'établissement d'accueil et le contexte technologique. Le second détaille la problématique et le cahier des charges du projet. Le troisième chapitre aborde l'analyse et la conception du système. Le quatrième expose la réalisation technique et le développement. Enfin, le dernier chapitre est consacré aux résultats, tests et à la validation du modèle.

Chapitre 1

Présentation de l'école et du contexte

1.1 Présentation de l'organisme d'accueil : l'EMSI

L'École Marocaine des Sciences de l'Ingénieur (EMSI) est un établissement d'enseignement supérieur privé reconnu par l'État, fondé en 1986. Pionnière dans la formation d'ingénieurs au Maroc, l'EMSI a pour mission de former des cadres de haut niveau capables de répondre aux exigences du marché du travail national et international.

Membre du réseau Honoris United Universities, l'EMSI propose plusieurs filières d'ingénierie, dont l'Ingénierie Informatique et Réseaux (IIR), qui vise à doter les étudiants de compétences solides en développement logiciel, en infrastructures réseaux, et en cybersécurité. Ce projet de fin d'année (PFA) s'inscrit directement dans cette démarche de mise en pratique des compétences acquises durant le cursus universitaire.

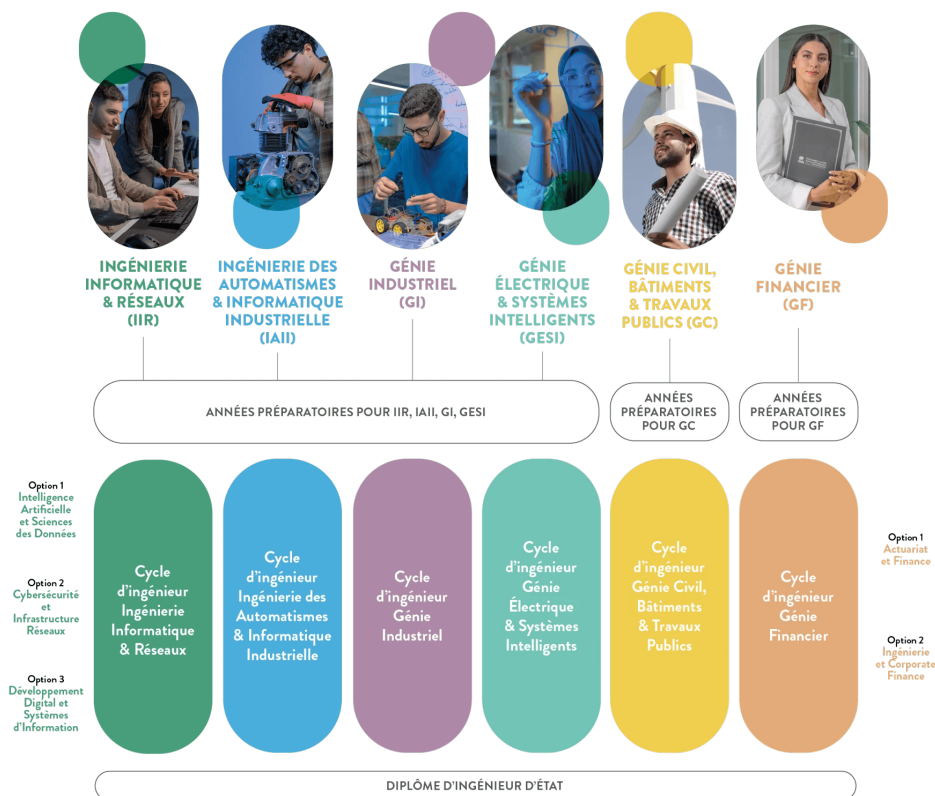


FIGURE 1.1 – Infrastructures de l'EMSI

1.2 Le phishing : définition et enjeux

Le phishing est une forme d'attaque par ingénierie sociale où l'attaquant envoie un email frauduleux imitant une entité de confiance afin d'inciter la victime à divulguer ses informations personnelles ou bancaires.

Les techniques courantes incluent :

- **L'usurpation de domaine** : le champ From affiche un domaine proche du réel (ex : paypa1.com).
- **L'utilisation de liens raccourcis** : services comme bit.ly pour masquer la destination réelle de l'URL.
- **La pression psychologique** : formules d'urgence pour précipiter la décision de la victime.

1.3 Technologies utilisées

Le projet repose sur une stack technologique moderne et robuste :

- **Django (Backend)** : Framework web Python de haut niveau, fournissant un ORM, un système de routage sécurisé et un moteur de templates.
- **PostgreSQL** : Système de gestion de base de données relationnelle puissant, utilisé pour stocker les rapports d'analyse.
- **Scikit-learn** : Bibliothèque de référence pour le Machine Learning en Python, utilisée pour l'entraînement et l'évaluation du modèle Random Forest.

Chapitre 2

Présentation du projet

2.1 Problématique

Les filtres antispam classiques, reposant sur des règles statiques, sont insuffisants face aux campagnes de phishing évolutives. La problématique centrale de ce projet se formule ainsi :

Comment développer un système capable de détecter automatiquement les emails de phishing en combinant l'analyse du contenu textuel et des caractéristiques structurelles, tout en garantissant un traitement rapide et sans rétention de données ?

2.2 Objectifs du projet

La solution "FakeFinder" vise à atteindre les objectifs suivants :

1. Traiter et analyser des emails bruts au format .eml en mémoire vive, sans jamais les sauvegarder sur un support physique (principe de "Zero Retention").
2. Évaluer le niveau de menace via un modèle de Machine Learning et générer un score de risque de 0 à 100.
3. Fournir une interface web intuitive pour l'utilisateur final.

2.3 Cahier des charges et fonctionnalités

Le système doit répondre à des exigences fonctionnelles claires :

- **F1 - Upload d'email** : Permettre le téléchargement sécurisé d'un fichier .eml.
- **F2 - Extraction de caractéristiques** : Parser l'email pour isoler les expéditeurs, le texte brut et les URLs.
- **F3 - Classification ML** : Interroger le modèle pré-entraîné pour obtenir une prédiction.
- **F4 - Restitution visuelle** : Afficher un code couleur clair (Vert/Safe, Orange/Suspect, Rouge/Danger).

2.4 Planning et gestion de projet (Méthode Agile)

La réalisation a été planifiée sur un cycle agile de 5 semaines, conformément au cahier des charges établi en début de projet.

TABLE 2.1 – Planning de réalisation du PFA (Avril - Mai 2026)

Période	Phase et Activités	Livrable
Du 06 au 12 Avril	Recherche documentaire, État de l'art, Conception (UML) et Maquettage.	CDC & Conception
Du 13 au 19 Avril	Développement du Backend Django et moteur de parsing email.	API d'analyse
Du 20 au 26 Avril	Intégration du modèle IA et développement Frontend.	Version Beta
Du 27 Avr au 03 Mai	Tests de détection sur datasets et validation des performances.	Rapport de tests
Du 04 au 10 Mai	Rédaction du rapport final de PFA et préparation de la soutenance.	Rapport Final

Chapitre 3

Analyse et conception

3.1 Analyse des besoins

La phase d'analyse a permis de déterminer l'architecture logicielle adéquate. Le système doit être hautement réactif : le traitement d'un email (upload, parsing, extraction, prédiction et réponse) doit s'effectuer en moins de 5 secondes. De plus, aucune donnée sensible issue du corps de l'email ne doit persister dans la base de données.

3.2 Conception UML

3.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation illustre les interactions possibles entre les acteurs (Visiteur, Administrateur) et le système FakeFinder.

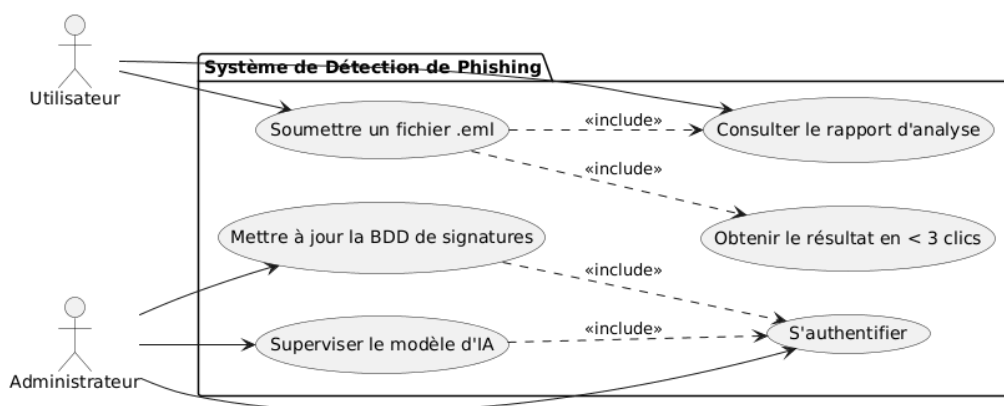


FIGURE 3.1 – Diagramme de cas d'utilisation du système

3.2.2 Modèle de données et Diagramme de classes

L'architecture de la base de données s'articule autour de l'entité ScanReport, qui consigne les résultats d'analyse sans conserver le fichier source.

TABLE 3.1 – Structure de l’entité ScanReport

Champ	Type Django	Description
risk_score	CharField	Verdict catégoriel : LOW / MEDIUM / HIGH.
score_value	IntegerField	Score numérique de confiance (0 à 100).
suspicious_urls	JSONField	Liste des URLs extraites du corps de l’email.
header_anomalies	JSONField	Métadonnées liées au domaine de l’expéditeur.

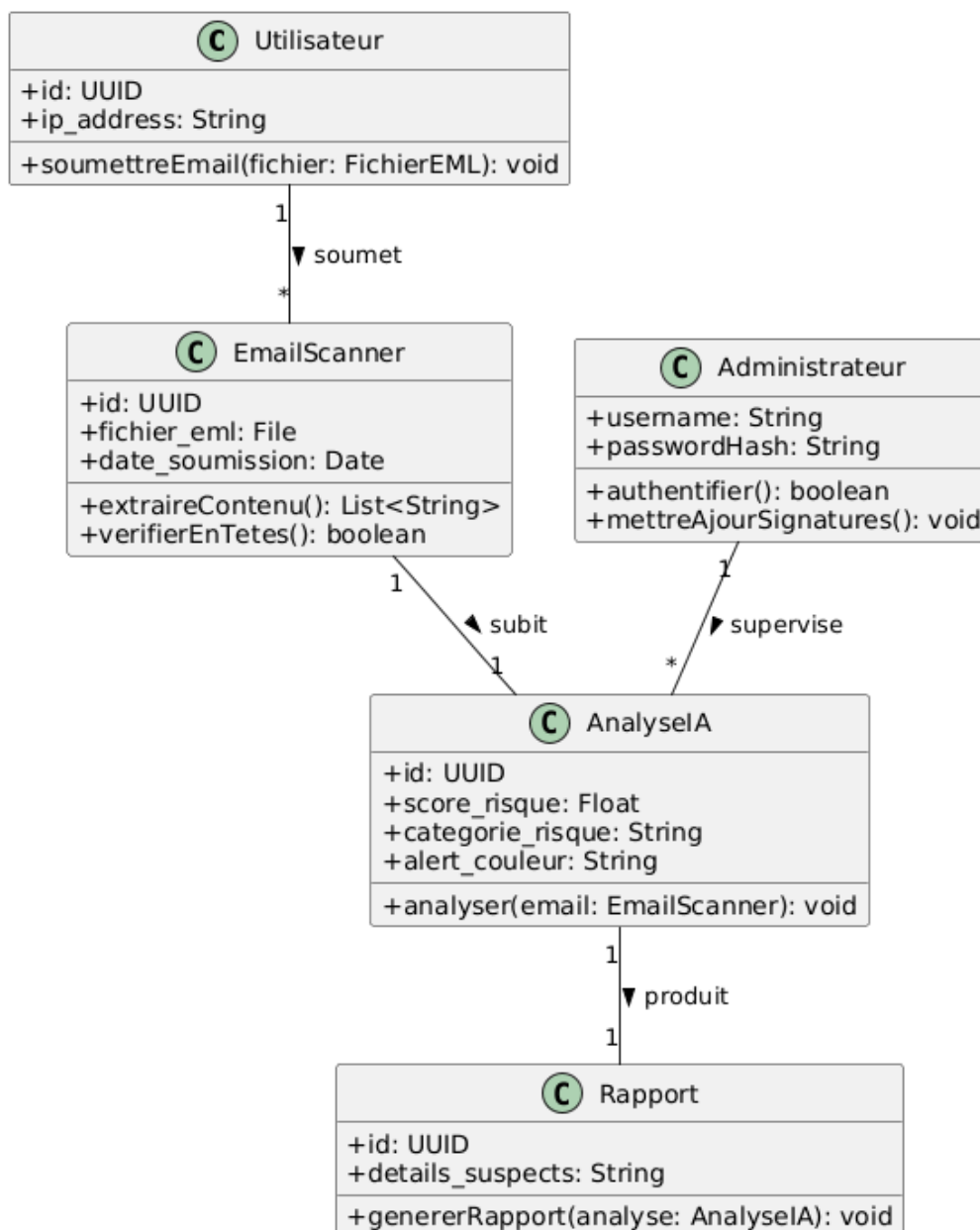


FIGURE 3.2 – Diagramme de classes du système Django

Chapitre 4

Réalisation et développement

4.1 Environnement de développement

Le projet a été développé en langage Python 3.12, en s'appuyant sur l'éditeur de code Visual Studio Code et le système de contrôle de version Git pour faciliter la collaboration.

4.2 Développement du moteur de Parsing (Backend)

L'une des étapes les plus critiques fut le développement du parseur d'emails capable de traiter des fichiers bruts en mémoire sans accès disque. Ce parseur extrait les URLs via des expressions régulières et décode le contenu HTML/Texte en contournant les problèmes d'encodage (UTF-8, ISO-8859-1).

4.3 Développement du modèle de Machine Learning

Le modèle prédictif repose sur l'algorithme **Random Forest**. L'extraction des caractéristiques ("Feature Engineering") combine deux approches :

1. **Features structurelles** : Comptage manuel d'éléments (nombre de liens, présence de raccourcisseurs, mots-clés d'urgence).
2. **Features textuelles (TF-IDF)** : Vectorisation de 20 000 unigrammes et bigrammes extraits du corps du texte.

Le modèle final a été exporté sous forme de fichier `.joblib` pour être interfacé avec les vues Django.

```

32 from sklearn.model_selection import train_test_split
33
34 # Make sure the project root is on sys.path so `ml.features` is importable
35 PROJECT_ROOT = Path(__file__).resolve().parent.parent
36 sys.path.insert(0, str(PROJECT_ROOT))
37
38 from ml.features import CombinedFeatureTransformer # noqa: E402
39
40
41 # — Dataset loading —
42
43 def load_dataset(csv_path: str):
44     """
45     Load the Kaggle Phishing Email CSV.
46     Returns X (array of email strings) and y (array of 0/1 labels).
47     """
48     df = pd.read_csv(csv_path)
49     print(f"Loaded {len(df)} rows. Columns: {list(df.columns)}")
50
51     # Auto-detect text column and label column
52     text_col = next(
53         (c for c in df.columns if "text" in c.lower() or
54          ("email" in c.lower() and "type" not in c.lower())),
55         df.columns[0],
56     )
57     label_col = next(
58         (c for c in df.columns if "type" in c.lower() or
59          "label" in c.lower() or "class" in c.lower()),
60         df.columns[1],
61     )
62     print(f"  text column : '{text_col}'")
63     print(f"  label column : '{label_col}'")
64
65     df = df[[text_col, label_col]].dropna()
66     df.columns = ["text", "label"]
67
68     # Normalise labels → 1 = phishing, 0 = safe
69     df["label"] = df["label"].str.strip().str.lower().map(

```

FIGURE 4.1 – Extrait de l’environnement de développement

```

FakeFinder > ml > features.py > ...
11
12 import re
13
14 import numpy as np
15 import scipy.sparse as sp
16 from sklearn.base import BaseEstimator, TransformerMixin
17 from sklearn.feature_extraction.text import TfidfVectorizer
18
19
20 # Keywords strongly associated with phishing emails
21 PHISHING_KEYWORDS = [
22     "verify", "account", "update", "confirm", "click here", "urgent",
23     "immediately", "suspended", "limited", "expires", "login", "password",
24     "bank", "credit", "debit", "security", "alert", "winner",
25     "congratulations", "free", "prize", "claim", "offer",
26     "dear customer", "valued customer", "paypal", "amazon",
27     "24 hours", "48 hours", "act now", "validate",
28     "unusual activity", "suspicious", "blocked", "restricted",
29     "your account", "reset", "temporary", "reactivate",
30 ]
31
32 # Common URL shortener domains used in phishing
33 SHORT_URL_PATTERN = re.compile(
34     r"bit\.ly|tinyurl\.com|goo\.gl|t\.co|ow\.ly|short\.link|rb\.gy|cutt\.ly",
35     re.IGNORECASE,
36 )
37
38 URL_PATTERN = re.compile(r"https?://[^\s<>\"']+ ", re.IGNORECASE)
39
40
41 class StructuralFeatureExtractor(BaseEstimator, TransformerMixin):
42     """
43     Extracts hand-crafted numerical features from raw email text.
44     Output is a sparse CSR matrix (one row per email, 10 columns).
45     """
46
47     FEATURE_NAMES = [

```

FIGURE 4.2 – Extrait de l’environnement de développement

4.4 Interface Utilisateur (Frontend)

Le frontend a été conçu pour être épuré. La page d'accueil propose une zone de dépôt ("Drag & Drop"). La page de résultat met en évidence le score de dangerosité par un code couleur (ex : rouge pour les menaces confirmées).

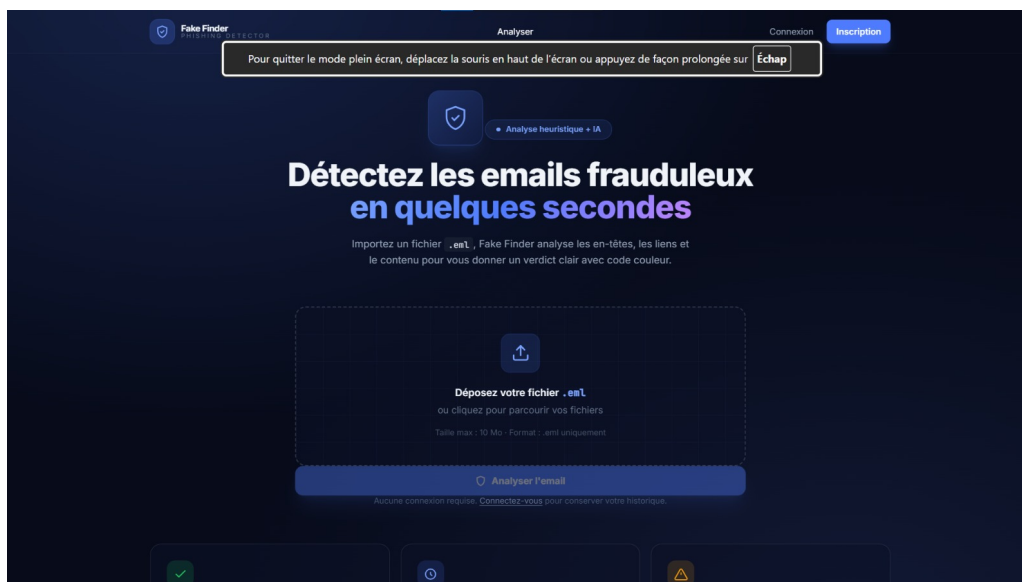


FIGURE 4.3 – Interface web : Zone d'upload sécurisée

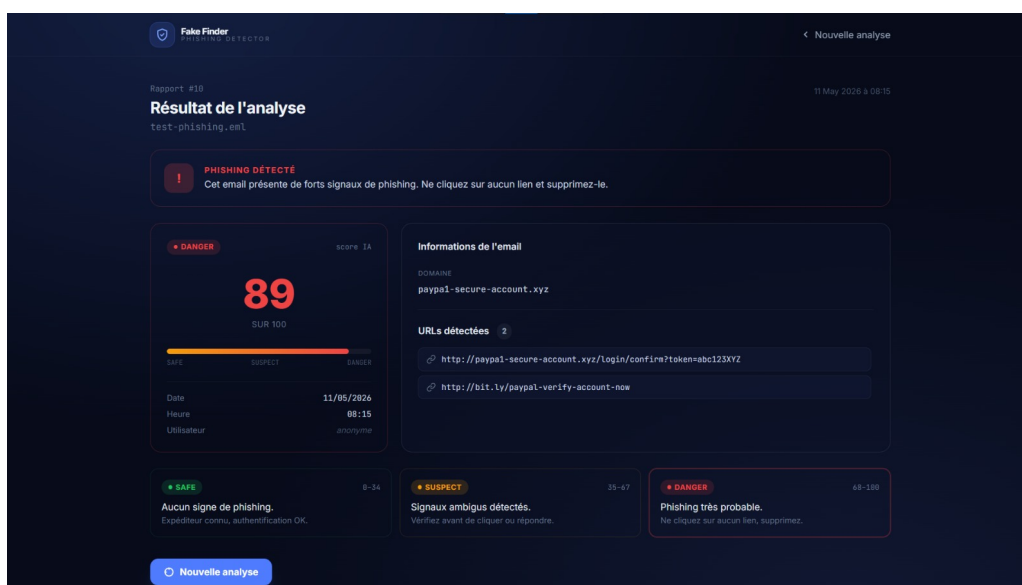


FIGURE 4.4 – Interface web : Zone de résultat du rapport

Chapitre 5

Résultats, tests et validation

5.1 Évaluation du modèle de Machine Learning

Le modèle a été entraîné sur le dataset public *Phishing Email Detection* composé d'environ 18 600 emails. Les résultats sur l'ensemble de test (20% des données) démontrent d'excellentes capacités de généralisation.

TABLE 5.1 – Rapport de classification du modèle Random Forest

Classe	Précision	Rappel	F1-score	Support
Emails Légitimes (Safe)	0,98	0,96	0,97	2 265
Emails Frauduleux (Phish)	0,94	0,97	0,96	1 462
Accuracy globale				97%

Avec une précision de 94% et un rappel de 97% sur la classe "Phishing", le modèle favorise la sécurité : il manque très peu de menaces réelles (peu de faux négatifs).

5.2 Tests fonctionnels et de robustesse de l'application

Plusieurs scénarios de test ont été exécutés sur l'application web finale pour valider le respect du cahier des charges :

- **Performance** : Le temps d'inférence et d'affichage est en moyenne de 1,2 seconde (largement inférieur aux 5 secondes exigées).
- **Sécurité** : Le téléchargement de fichiers intrus (ex : .pdf ou .exe) est bloqué nativement par le validateur de formulaire Django.
- **Résilience ML** : La soumission d'un email vide ou ne contenant que des images renvoie un score neutre sans provoquer d'erreur serveur (HTTP 500).

5.3 Analyse critique

Le système s'avère hautement fiable pour les emails rédigés en anglais, qui constituent l'immense majorité de notre jeu d'entraînement. Cependant, l'une des limites actuelles réside dans le traitement des campagnes de phishing ciblant spécifiquement la langue française, où le TF-IDF perd en efficacité à cause du vocabulaire non reconnu.

Conclusion et perspectives

Ce Projet de Fin d'Année nous a permis de concevoir, de développer et de déployer **Fake-Finder**, une solution complète alliant développement web et intelligence artificielle pour lutter contre le phishing.

Le choix d'un algorithme de type Random Forest couplé à une analyse TF-IDF s'est avéré particulièrement judicieux, permettant d'atteindre une précision globale de 97% sur un jeu de données conséquent. Par ailleurs, l'architecture Django adoptée garantit que l'application respecte les normes de confidentialité strictes, analysant les flux de données entièrement en mémoire vive sans les archiver.

Sur le plan pédagogique, ce projet fut une opportunité exceptionnelle de mettre en pratique les notions théoriques abordées à l'EMSI : génie logiciel, manipulation de bases de données, sécurité des systèmes d'information, et modélisation de données (Data Science).

Perspectives d'évolution : Afin d'améliorer cette solution, plusieurs axes de développement futurs sont envisageables :

1. **Support multilingue** : Enrichir le modèle avec des datasets francophones et hispanophones pour internationaliser la détection.
2. **Analyse dynamique des liens** : Ajouter un module vérifiant la réputation des URLs extraites en temps réel via des API tierces (ex : VirusTotal).
3. **Développement d'une API REST** : Permettre à des clients de messagerie lourds (Microsoft Outlook, Thunderbird) de s'interfacer directement avec notre moteur de prédiction sous la forme d'un plugin.

Bibliographie

Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *Proceedings of the anti-phishing working groups 2nd annual eCrime researchers summit*, 60-69.

Basnet, R., Sung, A. H., & Liu, Q. (2008). Rule-based phishing attack detection. *International Conference on Security and Management*, 22-28.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Django Software Foundation. (2024). *Django documentation - Version 6.0*. Repéré à [https ://docs.djangoproject.com/](https://docs.djangoproject.com/)

Hannousse, A., & Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection : An experimental study. *Engineering Applications of Artificial Intelligence*, 104, 104347.

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443-458.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Rout, S. (2022). *Phishing email detection dataset*. Kaggle. Repéré à [https ://www.kaggle.com/datasets/subhajou](https://www.kaggle.com/datasets/subhajou)

Sahingoz, O. K., Batur, E., Bulut, F., & Taskin, A. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345-357.

Verizon. (2024). *Data breach investigations report (DBIR)*. Repéré à [https ://www.verizon.com/business/resources](https://www.verizon.com/business/resources)

Annexe A

Extraits de code source

Script de parsing in-memory (utils.py)

```
1 import email
2 import re
3
4 def parse_eml_in_memory(file_obj):
5     raw_data = file_obj.read()
6     if isinstance(raw_data, str):
7         msg = email.message_from_string(raw_data)
8     else:
9         msg = email.message_from_bytes(raw_data)
10
11     sender = msg.get('From', '')
12     # Extraction dynamique des expressions regulieres
13     url_pattern = r'(https?://[\s>"\']+) '
14
15     # [...] Suite du traitement d'extraction [...]
16     return {
17         'sender_domain': sender,
18         'urls_found': True
19     }
```